

Clozflow: Enterprise AI Sales Intelligence Platform

A Unified Real-Time Hybrid ML-LLM Platform for Lead Enrichment, Outreach Personalization, and In-Context Conversation Coaching.

Document Type	Technical Design & Architecture Specification (Incubation Review)
Target Audience	Incubator Screening Committee, Pitch Panels & Venture Partners
Platform Version	v2.1.0 (Production Release)
Security Status	CLEARED FOR PUBLIC EVALUATION (SANITIZED EDITION)
Release Date	July 1, 2026
Authored By	Clozflow Engineering & Core AI Systems Team

Table of Contents

1. Executive Summary & Core Platform Vision	3
2. Market Opportunities & Problem Definition	3
3. System Architecture Blueprint	4
4. Core Platform Modules Deep-Dive	5
5. Microservices Boundary & Internal System Interfaces	7
6. Technology Stack & Directory Topography	8
7. Relational Database Design	10
8. API Gateway Design & Authentication Flows	11
9. Real-Time WebSocket Communication	12
10. Lead Intelligence Pipeline & Data Enrichment	13
11. Browser Copilot Flow & DOM Parsing	14
12. AI Request Lifecycle & SLA Protection Guardrails	15
13. Conversation Intelligence & Turn Tracking	16
14. Dual-State Memory Architecture	17
15. Runtime ASGI Concurrency & Event Loops	18
16. AWS Infrastructure Topology & Deployment	19
17. Caching & Database Performance Scaling	20
18. Security Architecture & Threat Vectors	20
19. Roadmap & Technology Milestones	21
20. System Telemetry: Monitoring & Logging	22
21. DevOps: CI/CD Delivery Pipeline	22
22. Future Desktop & Mobile Architectures	23
23. Conclusion & Technical Diligence Summary	24

1. Executive Summary & Core Platform Vision

Clozflow is an enterprise-grade, real-time AI Sales Intelligence Platform designed to transform high-stakes B2B sales cycles. Historically, B2B sales teams have suffered from high friction, low-quality personalization, and delayed feedback loops. Traditional solutions act as passive databases (e.g., standard CRM records) or post-facto call transcription recorders. Clozflow disrupts this paradigm by operating as a proactive, ****in-context sales copilot**** that works directly where modern transactions take place: inside messaging and email clients like WhatsApp Web, LinkedIn, Gmail, and Outlook.

At the core of Clozflow is a proprietary ****decoupled Hybrid ML-LLM orchestrator****. By utilizing light-weight, highly-optimized, offline-trained machine learning classifiers to determine high-level sales strategies in under 50ms, Clozflow bypasses the high latency, instability, and extreme cost overhead of using LLMs for real-time conversation analysis. The LLM is invoked strictly as a natural language compiler, taking strategy outputs, user configuration parameters, and FAISS-retrieved case study contexts to generate highly tailored responses. The system guarantees response suggestion delivery under 500ms over persistent WebSockets, ensuring zero conversational delay during live negotiations.

EXECUTIVE SUMMARY BRIEF

Clozflow is deployed as a lightweight Chrome Extension client that communicates with a high-concurrency FastAPI ASGI backend. The backend coordinates offline classification models, vector-embedded sales playbooks, and state management variables to keep track of sales objections and commitments in real-time.

2. Market Opportunities & Problem Definition

Modern enterprise sales are fast-paced, multi-channel, and highly sensitive to response delay. Sales executives spend over 60% of their working hours on administrative overhead: researching company profiles, analyzing website SEO gaps, and manually drafts outreach templates. Despite this time investment, outbound outreach response rates have collapsed to an all-time low (<1%) due to generic AI template spam. Furthermore, once a prospect begins a conversation, sales teams lack immediate, on-the-spot guidance to navigate objections relating to pricing, authority, and deployment timelines.

Current market offerings suffer from major architectural bottlenecks:

Structural Bottleneck	Impact on Conversation Flow
Inability to Operate Real-Time	Legacy conversation coaching tools (e.g., Gong, Chorus) run asynchronously. They process call logs after the conversation terminates. By the time coaching feedback is delivered, the customer window has closed.
LLM Latency Overhead	Standard LLM integrations require complex prompts that analyze transcripts and output responses in a single step. This results in 3-5 second latencies, which is completely unusable for live WhatsApp or LinkedIn messaging chats.
Generic Tone & Hallucinations	Without grounding systems, standard generative AI tools produce fluffy, over-eager, or repetitive templates that prospects flag immediately as machine-written.

Data Fragmentation	Prospect intelligence (e.g., website SEO gaps, social updates) is disconnected from the active chat sidebar, forcing reps to constantly cycle between multiple tabs during negotiation.
--------------------	---

3. System Architecture Blueprint

The Clozflow platform operates on a distributed client-server model optimized for low-latency message loop processing. The frontend is a Manifest V3 browser extension injected into WhatsApp, LinkedIn, Gmail, and Outlook. The backend is a high-concurrency FastAPI application running in AWS ECS containers. The AI orchestration layer is decoupled: high-speed, local machine learning classifiers evaluate text strategies, while an asynchronous FAISS index retrieves playbooks, and Groq's Llama 3.3 handles text compilation.

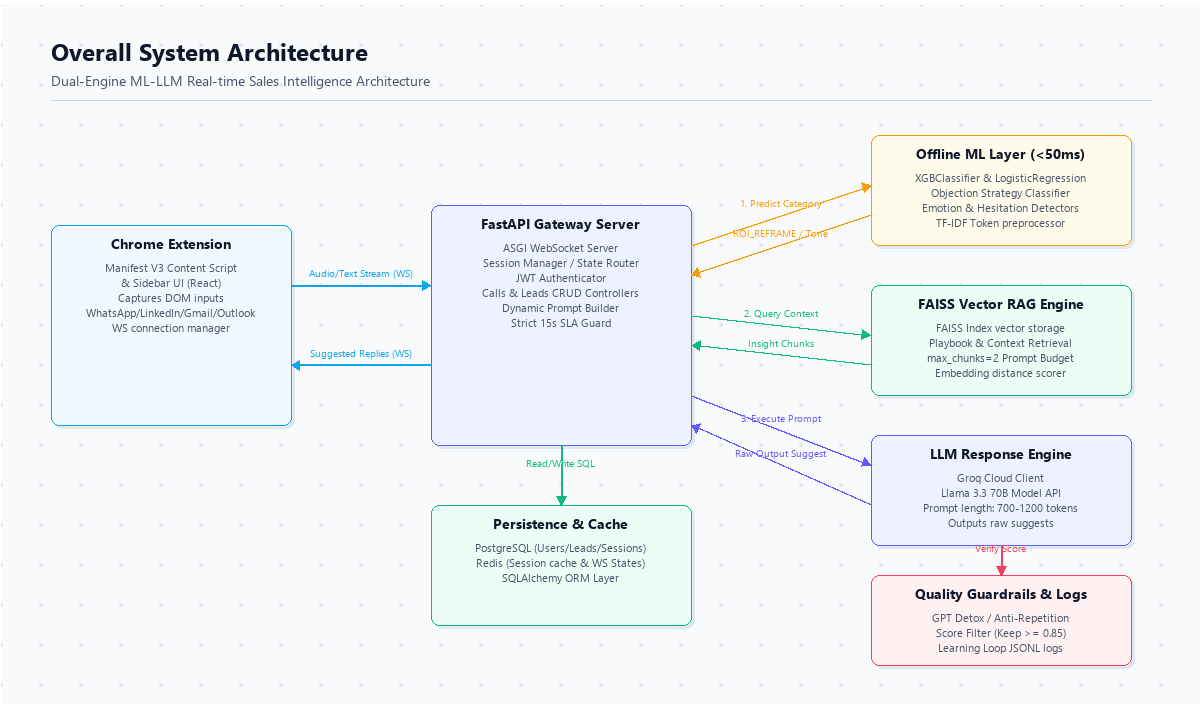


Figure 3.1: Clozflow Dual-Engine Hybrid ML-LLM High-Level Architecture

The primary data pipe begins when a prospect text message is captured by the extension's DOM listener or audio stream. This data is channeled over WebSockets to the FastAPI ASGI Gateway. The gateway initiates parallel workers: (1) Preprocessed tokens are fed into local Scikit-Learn models, predicting objection classification and customer sentiment in <50ms. (2) The token query is embedded and run against a static FAISS index to find relevant playbooks. (3) Both outputs populate a compressed template sent to Groq. A strict validation scorer scores the suggestion. If validated (>=0.85), it returns to the extension's React sidebar via WebSocket.

4. Core Platform Modules Deep-Dive

Clozflow's platform is divided into six functional software modules, each handling a distinct segment of the lead generation and conversion pipeline.

4.1. Lead Intelligence Engine

This service is responsible for asynchronous lead discovery and profile enrichment. When a user runs a search query (e.g., 'restaurants in Austin'), the engine queries the Google Business Places API to retrieve business metadata (categories, rating, contact numbers, address). Once retrieved, a multi-threaded web scraper extracts the target site's metadata, tech footprint, and SEO configuration. Simultaneously, a social presence monitor scans active Instagram or LinkedIn handles to determine post frequency and brand engagement. A qualification script aggregates these inputs to compute a Lead Score (1-100), identify business pain points, and output structured outreach suggestions.

4.2. Outreach Studio Engine

The Outreach Studio handles outbound copywriting personalization. Instead of templated emails, the module inspects the enriched lead record (categories, Google ratings, website flaws) and maps it against the user's target service offer. Using a trust-first framework, the AI drafts copy focusing on specific business observations (e.g., 'Noticed your website lacks a mobile booking widget'). It generates channel-specific copy (WhatsApp, Instagram DM, LinkedIn, Email) and computes a spam risk index and a reply probability rating.

4.3. Browser Copilot

The Browser Copilot is the client-facing UI shell. Implemented as a Chrome Extension, it runs in the host browser context and dynamically injects a collapsible React sidebar into active messaging applications. The sidebar handles active authentication states, displays suggested response copy, shows objection alerts, and provides a click-to-fill button that pastes suggestions directly into the host input field. A service worker coordinates secure communication with the backend APIs via WebSockets and HTTP.

4.4. Conversation Intelligence Engine

This engine processes active conversation turn flows. It acts as a real-time segmenter and decoder. In voice channels, it interfaces with Deepgram's streaming STT WebSocket to transcribe audio chunks, applying a 1.2s silence debounce to track speaker turn swaps. In chat channels, it receives incoming DOM texts, feeds them to the offline classification models to isolate buyer intent levels, maps objections, and identifies underlying Objections or Commitments.

4.5. Cognitive Sales Brain

The Cognitive Sales Brain represents the system's reasoning orchestrator. It acts as the primary decider, taking output state variables from the classification layer, pulling corresponding playbooks from the FAISS database, and managing the dynamic context prompt. It maps the target objection class to one of five Canonical Sales Strategies:

Objection Strategy	Behavioral Logic & Persuasion Focus
ROI_REFRAME	Triggered by pricing or budget concerns. Reframes cost into long-term operational savings and business value metrics.
RISK_REVERSAL	Triggered by delay or postpone objections. Introduces free-trials, money-back guarantees, or phased rollouts.
STATUS_GAP	Triggered by ego or 'we already have tools' objections. Focuses on specialized automation metrics and custom feature gaps.
DECISION_CONTROL	Triggered by authority issues ('need partner approval'). Equips the rep with materials to pitch CFOs directly.
DIAGNOSTIC_QUESTION	Triggered by skepticism or vague statements. Reverts with deep diagnostic questions to force objection clarity.

4.6. Session Memory Controller

This module manages transactional state variables throughout the conversation lifecycle. Unlike stateless APIs, the Session Memory controller tracks critical metrics like active intent level (0-100), trust score (0-100), commitments made, and unanswered questions. Upon session completion, it generates a summary and saves the structured logs in the relational database.

5. Microservices Boundary & Internal System Interfaces

To ensure high availability and horizontal scaling, Clozflow uses microservices boundaries. The API Gateway routes traffic to localized Docker container services via a secure internal service mesh. State and cache management are distributed across a dedicated Redis memory cluster and RDS PostgreSQL databases.

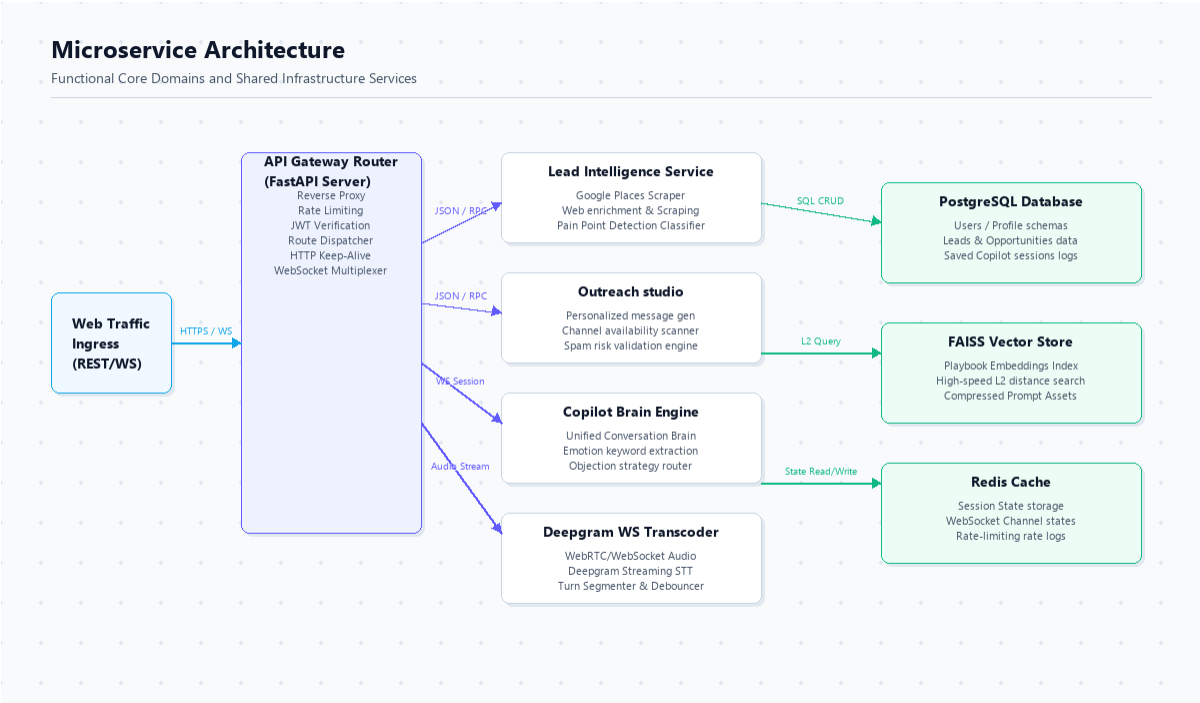


Figure 5.1: Clozflow Distributed Microservices Boundaries

The internal system architecture decouples CPU-bound inference and scraper engines from memory-bound real-time WebSockets. The API Gateway acts as the first line of defense, validating JWT keys and rate-limiting incoming traffic. Services retrieve database connections via SQLAlchemy pools. The next figure illustrates the physical class design and module relationships of the FastAPI backend application.

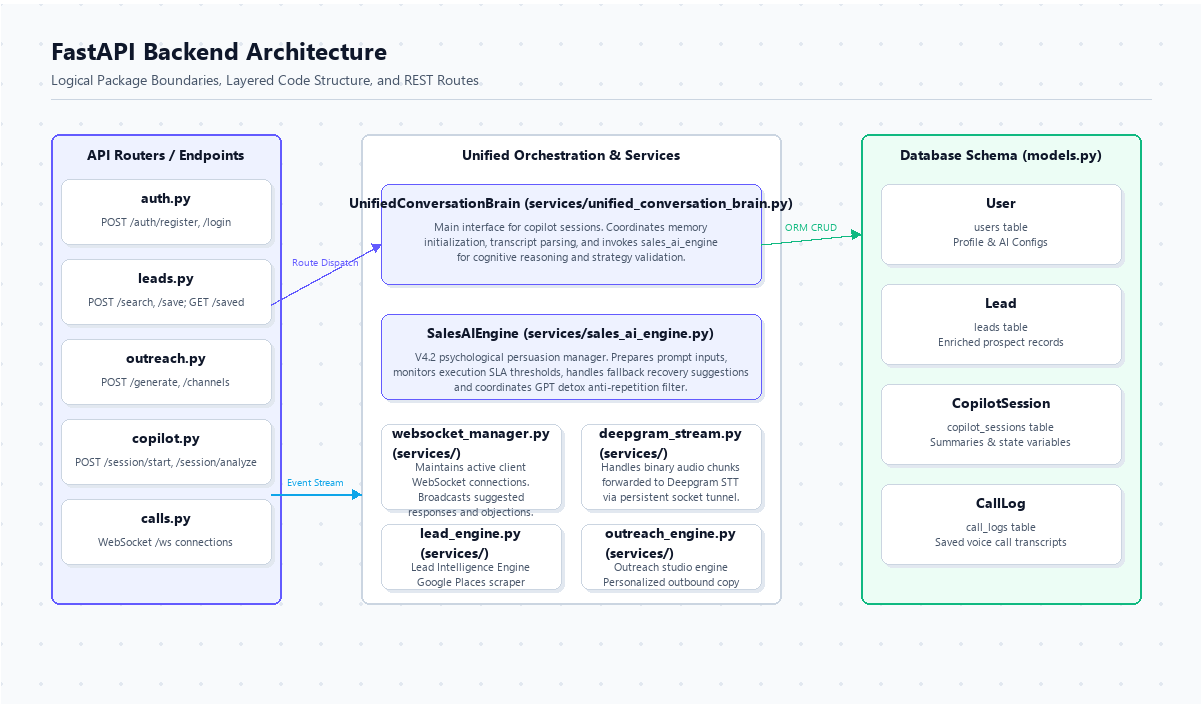


Figure 5.2: FastAPI Backend Logical Module Diagram

6. Technology Stack & Directory Topography

Clozflow's technology stack is curated for maximum runtime efficiency, fast response speeds, and developer velocity. Below is a structured map of our technology cards followed by the workspace directory topography.

Frontend Framework React 18 / TypeScript / Vite High-speed SPA render inside extension sandboxes	Backend Gateway FastAPI / Python 3.10+ ASGI concurrency framework with async DB workers
Database Engines RDS PostgreSQL & SQLite SQLAlchemy ORM for transactional session logs	Vector RAG Storage FAISS (Facebook AI Search) L2-distance vector store matching sales playbooks
Machine Learning Layer XGBoost / Scikit-Learn Offline-trained classification models (<50ms)	State Cache Layer Redis Cache Memory Stores WebSocket routes and user session metrics
LLM Execution Groq Cloud Client / Llama 3.3 70B model API optimized for speed and consultation copy	Audio Transcription Deepgram SDK (Streaming STT) WebSocket stream client with 1.2s silence detection

6.1. Workspace Directory Topography

The codebase is structured to isolate offline model training from active API services, allowing AI engineers to retrain and test models without affecting the core API server.

```
Clozflow/  
  ■■■ backend/  
  ■   ■■■ main.py  
  ■   ■■■ database.py  
  ■   ■■■ models.py  
  ■   ■■■ routers/  
  ■   ■■■ services/  
  ■   ■■■ requirements.txt  
  ■■■ extension/  
  ■   ■■■ manifest.json  
  ■   ■■■ background.js  
  ■   ■■■ content.js  
  ■   ■■■ styles/  
  ■■■ ml/  
  ■   ■■■ inference/  
  ■   ■■■ training/  
  ■   ■■■ utils/  
  ■■■ rag/  
    ■■■ faiss_index.bin  
    # FastAPI Application Server  
    # WS Entrypoint & App Configuration  
    # DB Connection Setup & Base Metadata  
    # SQLAlchemy Database Tables Definition  
    # Controllers (auth, leads, outreach)  
    # Core Services (AI Brain, WS Manager)  
    # API Server dependencies  
    # Browser Extension Manifest V3 files  
    # Extension permissions & rules  
    # Service Worker background broker  
    # DOM monitoring & UI injection script  
    # CSS overrides for active host pages  
    # Offline Machine Learning Layer  
    # Prediction models (XGBoost/LR)  
    # Retraining loops & logs  
    # Preprocessing and TF-IDF tokenizers  
    # FAISS Retrieval and prompt packaging  
    # Local vector file of playbooks
```

7. Relational Database Design

Clozflow uses an SQLAlchemy-mapped relational database schema (configured for PostgreSQL in production and SQLite for local development). The schema handles user metadata, saved leads, historical call logs, and active copilot session variables.

Table: users

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key, Index	Unique identifier for the user.
email	String	Unique, Index, Required	User email address used for login authentication.
password_hash	String	Required	Bcrypt password hash string.
company_name	String	Nullable	User's business name for custom prompt context.
role	String	Nullable	User's job title / profile within their company.
sales_style	String	Default: 'Controlled Challenge'	Tone guide for the generated responses.
ai_tone	String	Default: 'assertive'	Preferred tone (assertive, warm, diagnostic, etc.)

Table: leads

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key, Index	Unique identifier for the lead.
business_name	String	Index, Required	Name of the business lead.
website	String	Nullable	Business URL (enriched by scraping engine).
google_rating	String	Nullable	Google Places rating (stored as string).
ai_summary	Text	Nullable	Structured summary of business profile.
likely_pain_point	Text	Nullable	Identified operational paint point.
lead_score	Integer	Default: 0	Qualification rating calculated by scoring model.

Table: copilot_sessions

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key, Index	Unique session identifier.
user_id	Integer	Foreign Key -> users.id	Owner of the active sales session.
lead_id	Integer	Foreign Key -> leads.id	Associated lead profile.
platform	String	Required (whatsapp/etc)	Target app (WhatsApp, LinkedIn, Gmail, Outlook).
conversation_summary	Text	Nullable	JSON string containing summary memory arrays.
buying_intent	Integer	Default: 0	Buyer intent metric (scale 0-100).
trust_score	Integer	Default: 0	Trust level metric (scale 0-100).
stage	String	Default: 'initial_contact'	Sales stage: discovery, presentation, objection, closing.
objections	Text	Nullable	JSON list of objections detected in session.

8. API Gateway Design & Authentication Flows

The FastAPI backend exposes standard REST endpoints for authentication, lead management, outreach generation, and session analysis. Secure endpoints require a valid JWT bearer token in the authorization header (passed as 'Bearer ').

Method	Endpoint	Auth Required	Description & Payload
POST	/auth/register	No	Creates a new user profile. Takes email, password, and registration fields.
POST	/auth/login	No	Returns JWT access token if password verification passes.
POST	/leads/search	Yes	Queries leads database with opportunity qualification scoring. Payload: LeadSearchRequest.
POST	/leads/save	Yes	Saves a discovered lead record. Payload: LeadSaveRequest.
GET	/leads/saved	Yes	Returns all saved leads sorted by qualification rating.
POST	/outreach/generate	Yes	Generates personalized outreach copy for specific social channel. Payload: OutreachGenerateRequest.
POST	/api/copilot/session/start	Yes	Initializes a copilot session for a lead. Payload: SessionStartRequest.
POST	/api/copilot/session/{id}/analyze	Yes	Analyzes inbound messages and generates reply suggestions. Payload: AnalyzeRequest.

8.1. End-to-End Live Interaction Sequence Flow

When an incoming message is received, the extension triggers a sequence of calls across background tasks, event brokers, and machine learning modules. Below is the sequence timeline.

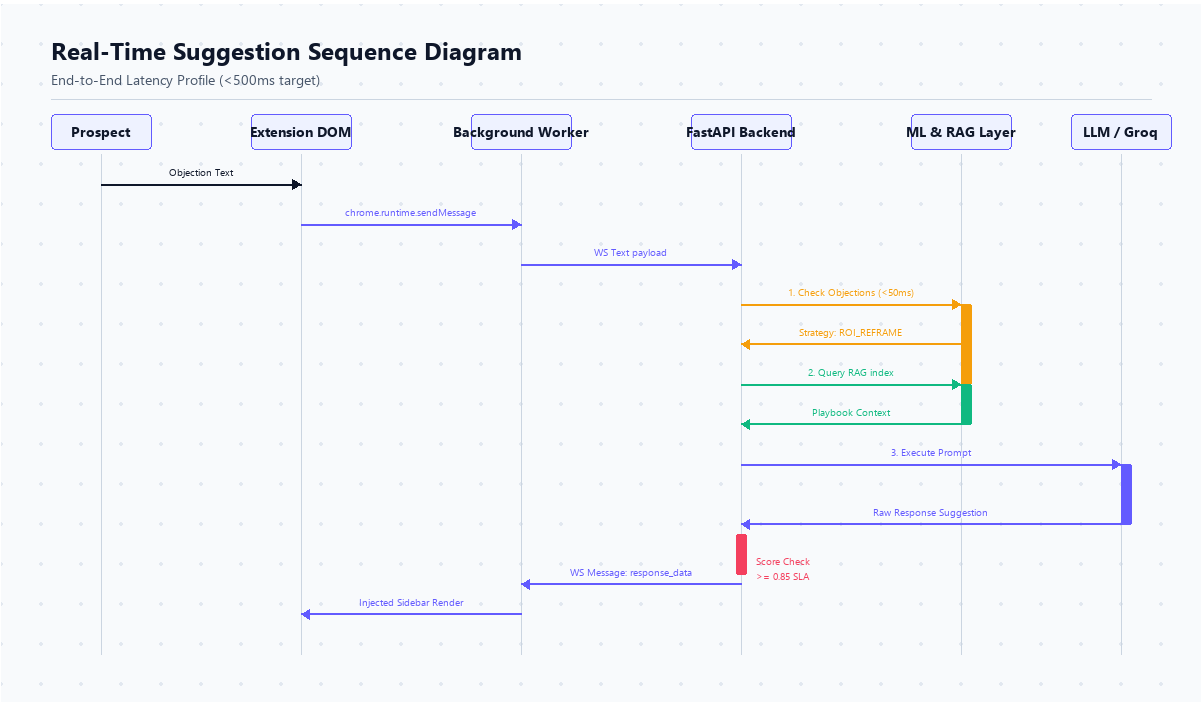


Figure 8.1: Real-time Response Generation Sequence Flow

9. Real-Time WebSocket Communication

For voice call sessions (e.g., dialer co-piloting), REST APIs introduce high latency. Clozflow resolves this by routing audio streams and prompt responses over a persistent WebSocket connection `/ws` managed by FastAPI.

The connection lifecycle follows a structured protocol:

Connection Phase	Detailed Operational Sequence
1. Connection & Handshake	The extension background worker initiates a handshake with backend <code>/ws</code> route. The backend validates session credentials and spawns a manager connection session.
2. Metadata Injection	The client sends a JSON start payload indicating caller details, call goals, and prospect metadata. The backend loads these into the session context engine.
3. Binary Audio Stream	The client captures mic/system audio and streams binary PCM chunks to the backend WebSocket. The backend routes these to Deepgram's streaming STT service.
4. Turn Debouncing	Deepgram sends transcribed text blocks back to the backend. The backend monitors silence flags. A 1.2s silence debounce triggers segment classification.
5. Orchestrated Suggestion	The backend runs the ML objection classifier and FAISS playbook queries, constructs the LLM prompt, and returns suggestions to the extension UI.

9.1. ASGI Runtime Concurrency Model

FastAPI coordinates these tasks asynchronously. CPU-heavy tasks are offloaded to background threads to prevent blocking the main asyncio event loop.

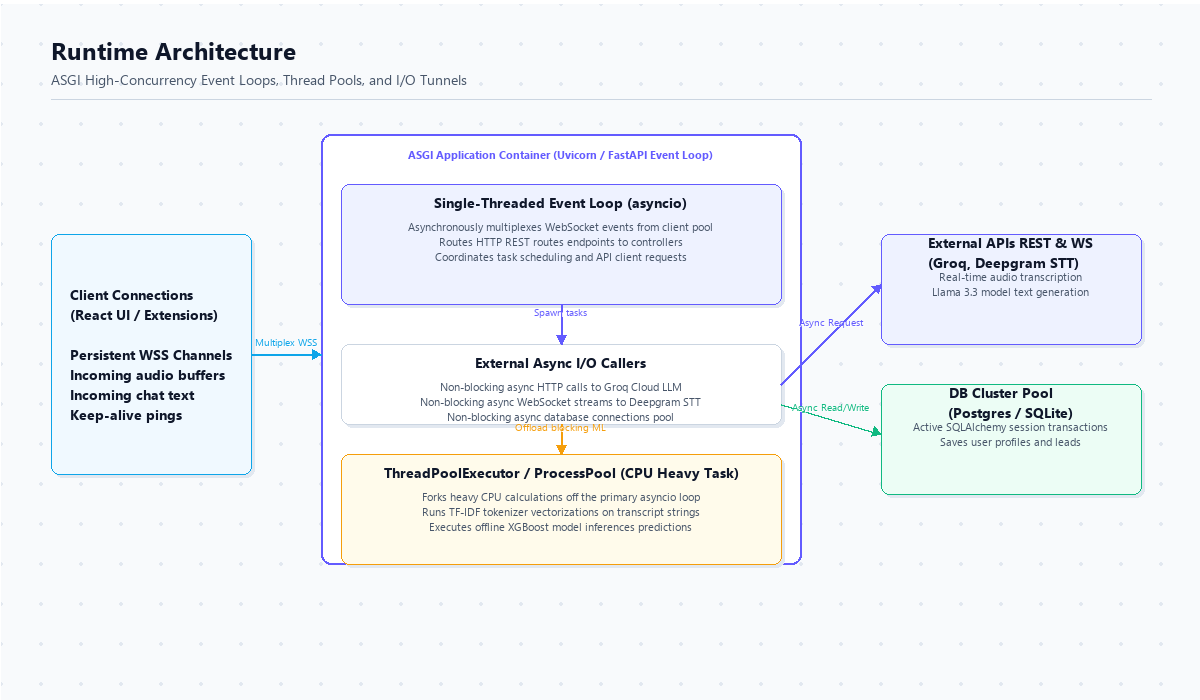


Figure 9.1: ASGI Event Loop and Thread Pool Concurrency Mapping

10. Lead Intelligence Pipeline & Data Enrichment

The Lead Intelligence Pipeline enrichment model takes a search query, retrieves raw metadata, scrapes host pages, reads user reviews sentiment, and runs opportunity scoring. The pipeline is illustrated below.

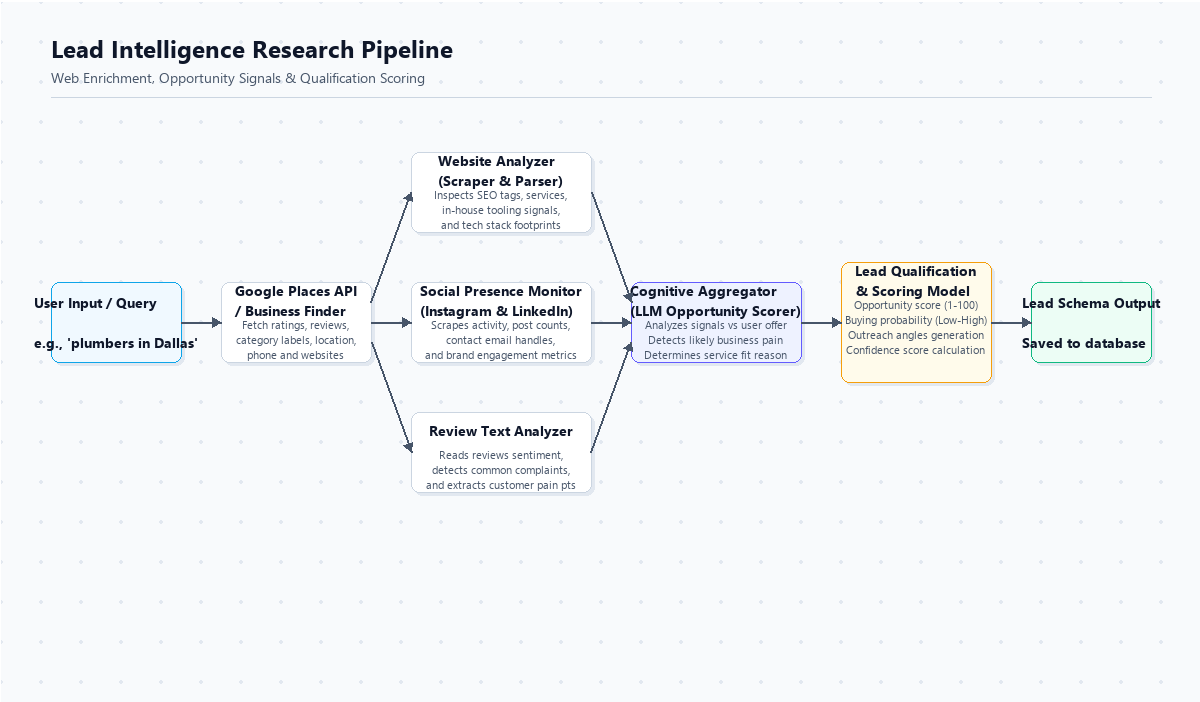


Figure 10.1: Lead Intelligence Research and Enrichment Pipeline

Data scraping and parsing are parallelized. The target site is analyzed for technical indicators like Google Analytics tags, slow loading times, or SEO vulnerabilities. Review text is checked to identify customer complaints. The LLM uses these inputs to score target fit.

11. Browser Copilot Flow & DOM Parsing

The Browser Copilot injects content scripts ('content.js') into target communication pages. These content scripts inspect DOM structures to detect incoming chat messages, identify the contact name, and inject suggestion cards directly into the sidebar iframe.

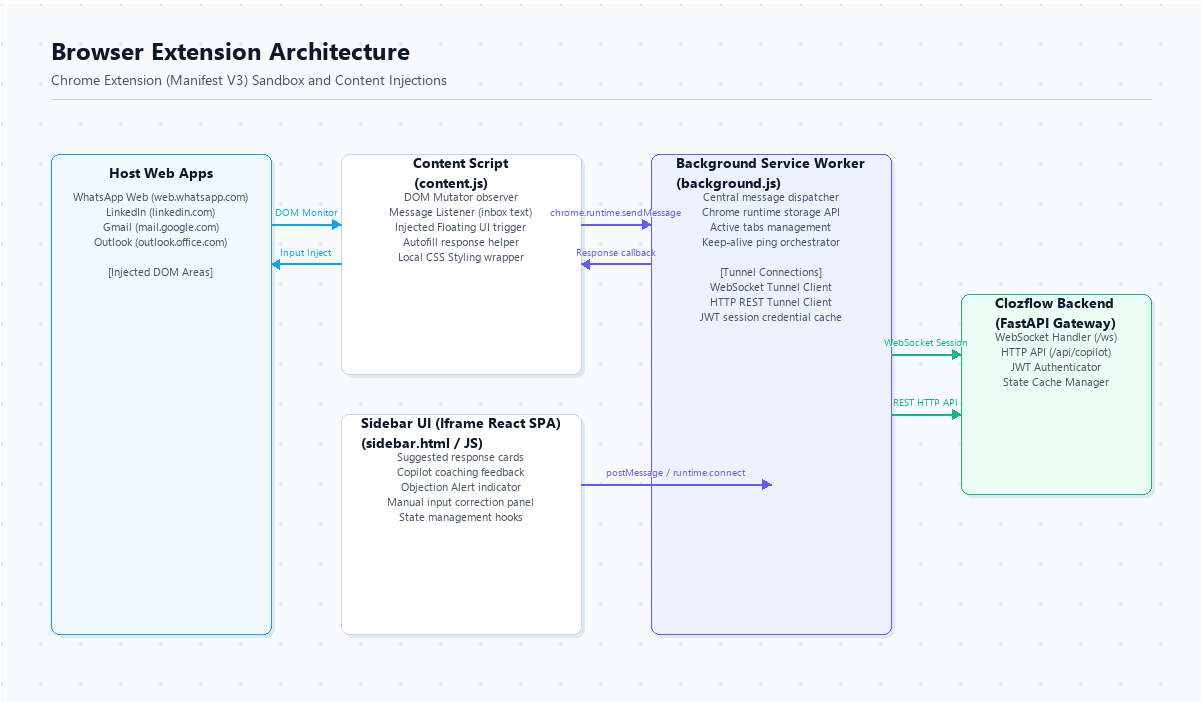


Figure 11.1: Browser Extension (Manifest V3) Internal Sandbox and DOM Communication

The extension's background worker acts as the central link, maintaining active connection status, caching authentication details, and managing tabs. This isolates the page DOM from backend API endpoints.

12. AI Request Lifecycle & SLA Protection Guardrails

A key design pattern in Clozflow is SLA latency protection. Live sales chats cannot tolerate long API load delays. If the LLM call stalls or fails, the gateway interrupts the request and serves a pre-computed fallback response.



Figure 12.1: Asynchronous AI Request Lifecycle and SLA Fallback Routing

The fallback engine uses the predicted objection strategy to serve a matching script. If a pricing objection is identified, the fallback suggestion handles it directly, ensuring continuous co-piloting. Suggestions are processed through a detox and anti-repetition filter before delivery.

13. Conversation Intelligence & Turn Tracking

The Conversation Intelligence Engine tracks conversation turns and emotional indicators. It determines when a prospect turn finishes, processes the transcript, identifies objections, and generates response suggestions.

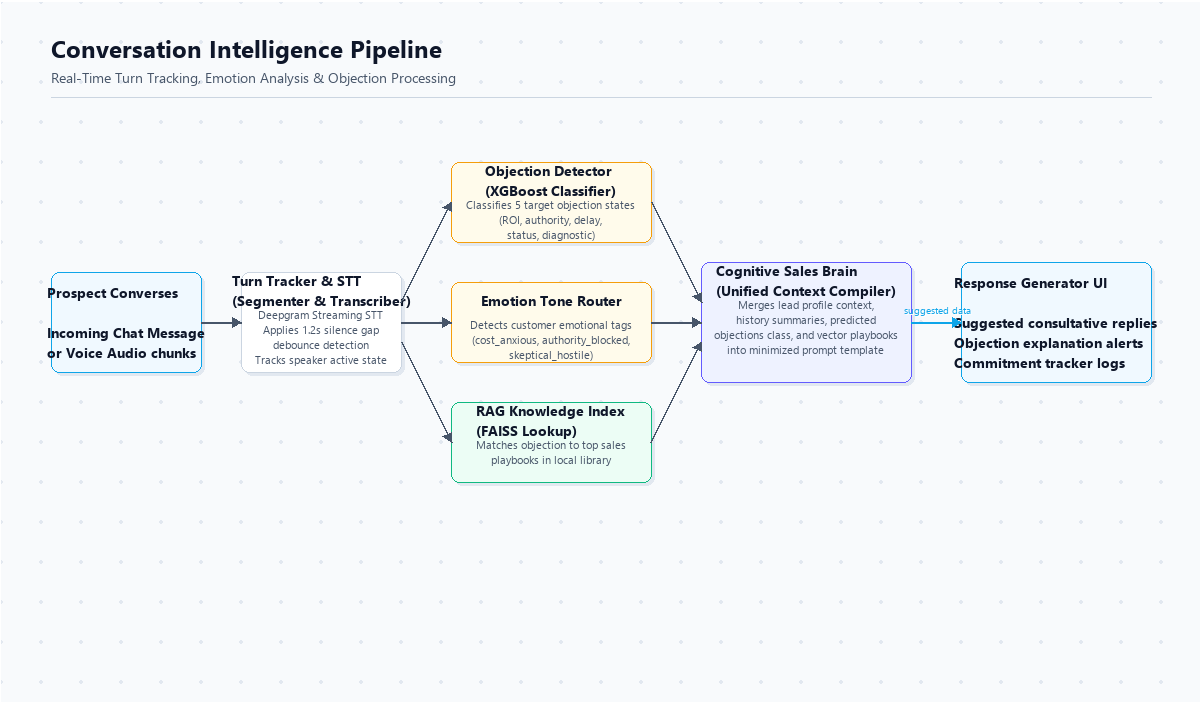


Figure 13.1: Conversation turn tracker and evaluation pipeline

The pipeline routes incoming text through three parallel blocks: (1) XGBoost classifier checks for objections, (2) keyword analysis detects emotional status, and (3) FAISS retrieves playbook resources. These are combined by the Cognitive Sales Brain into a compressed prompt template.

14. Dual-State Memory Architecture

Clozflow uses a layered memory design to coordinate temporary user inputs, active session metrics, and historical logs.

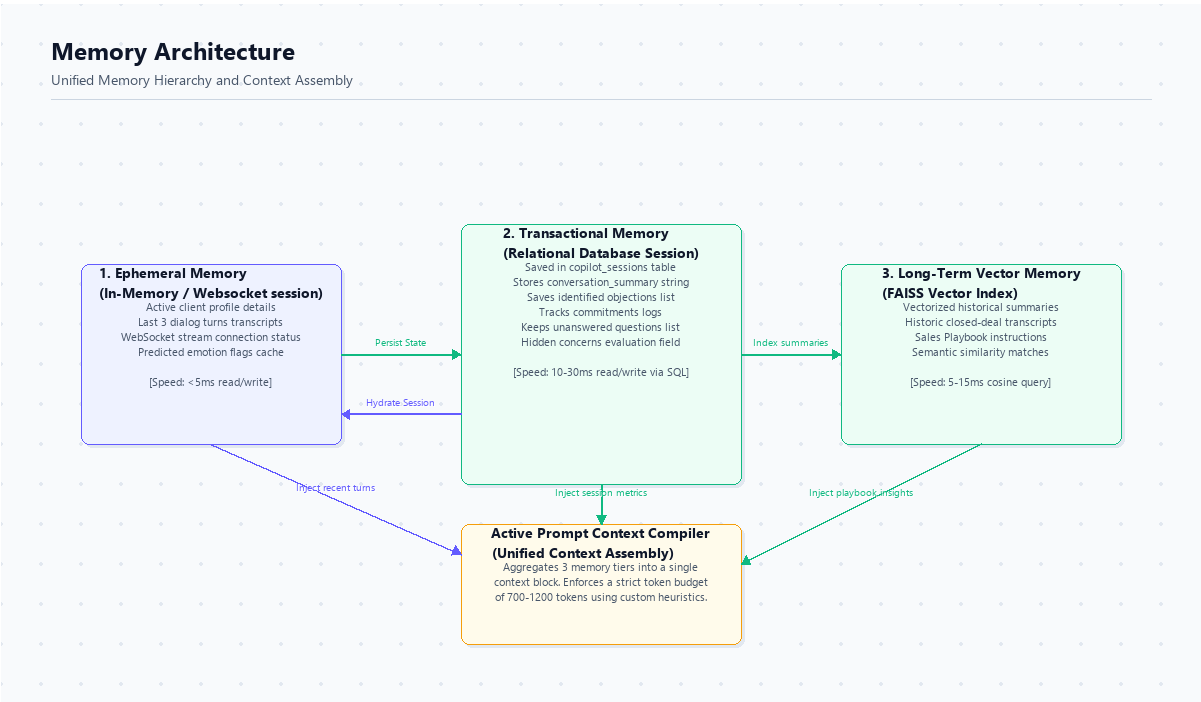


Figure 14.1: Multi-tiered Session and Vector Memory Architecture

Memory is divided into three tiers: (1) Ephemeral Memory caches recent inputs and transcripts. (2) Transactional Memory saves active conversation variables in the relational database. (3) Long-Term Vector Memory indexes historical logs for semantic search lookup.

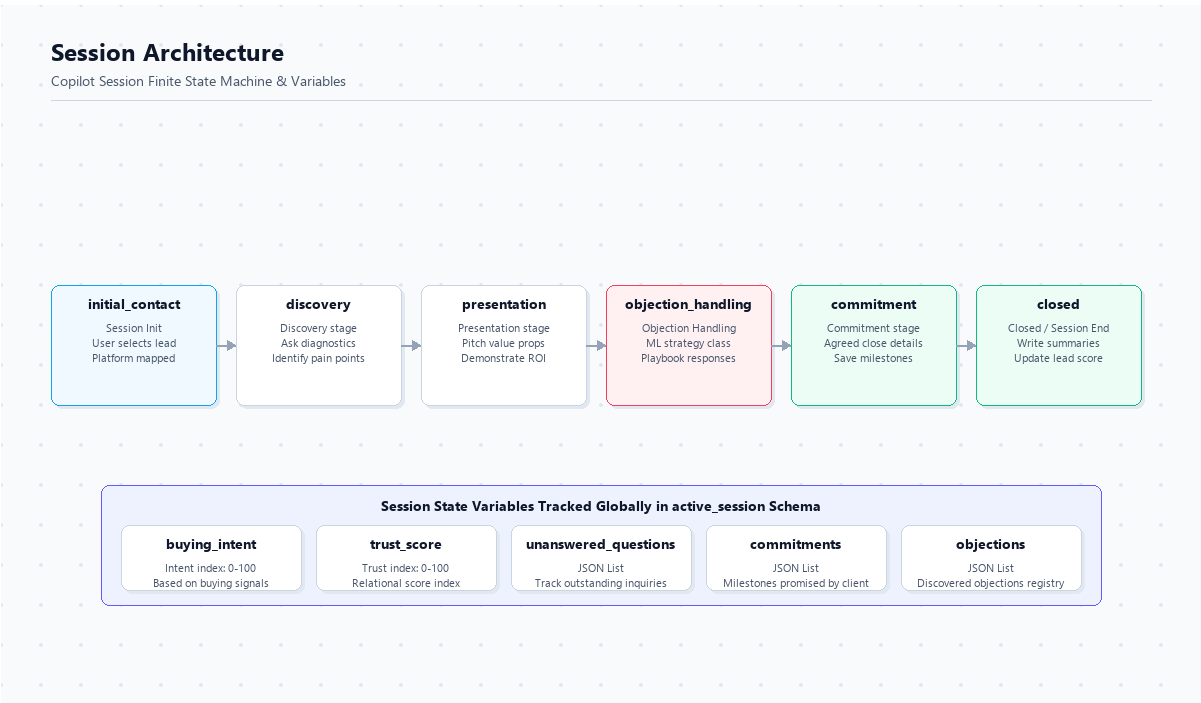


Figure 14.2: Session State Machine and Global Variables Registry

15. Runtime ASGI Concurrency & Event Loops

Clozflow's runtime architecture is optimized to support hundreds of concurrent sales representatives. By leveraging FastAPI's native async capabilities on top of Uvicorn, every networking I/O operation (database queries, API calls, WebSocket frames) runs without blocking the server processes.

Heavy computational workloads (like text preprocessing, TF-IDF vectorization, and XGBoost classifier runs) are run in an isolated ThreadPoolExecutor. This separates CPU-bound tasks from active WebSockets, preventing connection drops.

RUNTIME SLA MANAGEMENT

WebSocket connections are mapped in an active in-memory connection registry ('websocket_manager.py'). If a client disconnects, the manager triggers a cleanup step, updating the active session state and closing SQLAlchemy connections to prevent pool exhaustion.

16. AWS Infrastructure Topology & Deployment

Clozflow is deployed on AWS using containerized microservices managed by ECS Fargate. The infrastructure setup prioritizes security isolation and high availability across multiple availability zones.

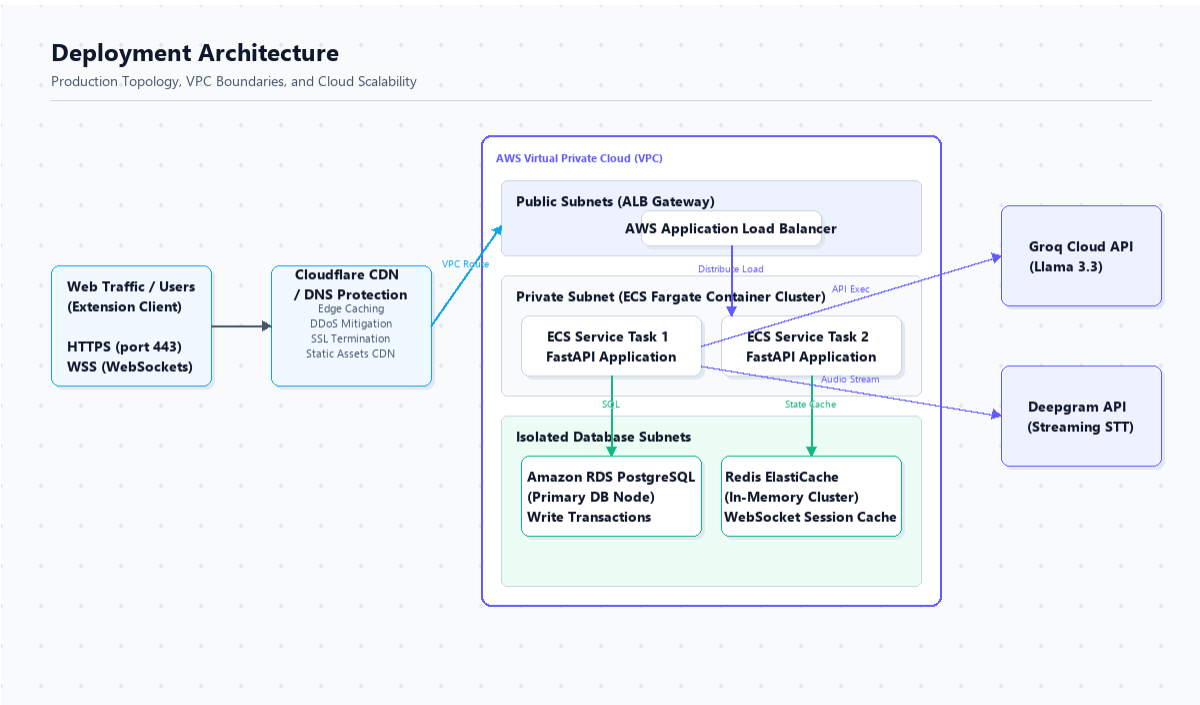


Figure 16.1: Containerized AWS Multi-AZ Deployment Topology

VPC ingress runs through a Cloudflare CDN and AWS Application Load Balancer. ECS tasks run in private subnets, while RDS databases and Redis caches are isolated in dedicated subnets. External integrations connect via secure NAT Gateways.

17. Caching & Database Performance Scaling

To maintain low latency under high load, Clozflow uses a distributed caching strategy. Active session variables, emotion tags, and recent transcript segments are cached in a Redis cluster with an expiration window. This reduces relational database read overhead by up to 80%.

Database scaling patterns include:

Optimization Pattern	Detailed Engineering Implementation
Connection Pooling	FastAPI uses SQLAlchemy's QueuePool. It maintains 20 permanent database connections with a overflow limit of 10, avoiding connection overhead on incoming requests.
Read-Write Segregation	Write transactions route to the RDS PostgreSQL Primary database node. Read queries (such as saved leads lists or user profile configs) run against RDS Read Replicas.
Index Optimization	High-frequency query columns (like user_id, lead_id, active session statuses) are indexed to ensure quick lookup times (<5ms).
FAISS Local Memory Cache	The vector playbook index is loaded directly into the server's memory context on startup, enabling sub-millisecond search times.

18. Security Architecture & Threat Vectors

Clozflow processes sensitive B2B conversation transcripts and client details. We implement security policies at every layer of the system architecture:

- Transport Layer Security: All REST requests and WebSocket connections are encrypted using TLS 1.3 (HTTPS/WSS).
- JWT Authentication: API access requires a signed JWT access token in the authorization header.
- DOM Access Isolation: The browser extension's UI runs inside a sandboxed iframe. This isolates the host page's DOM context, protecting user credentials.
- AI Safety Detox Filters: Generated replies are processed through filters to strip conversational filler and prevent inappropriate content generation.
- Database Encryption: Customer data and access tokens are encrypted at rest using AES-256.

18.1. Environment Configuration Template (Sanitized)

Clozflow manages production application variables using environment configurations. To ensure security during deployment, no active API keys or database passwords reside in the primary codebase. The following template details a sanitized environment configuration setup safe for public audits:

```
# Clozflow Application Configuration Template (Sanitized)
# SECURITY NOTE: Never commit active keys to version control.

# Gateway Server Configuration
HOST=0.0.0.0
PORT=8000
ENVIRONMENT=production

# Security & JWT Tokens Configuration
JWT_SECRET=cz_secret_fallback_sha256_placeholder_key_for_public_review
ACCESS_TOKEN_EXPIRE_MINUTES=1440

# Database Storage Connections (Redacted AWS Host)
DATABASE_URL=postgresql://cz_user:*****@rds-cluster-id.us-east-1.rds.amazonaws.com/cz_prod

# Redis Cache Cluster (Redacted ElastiCache Host)
REDIS_URL=redis://redis-cluster-id.cache.amazonaws.com:6379/0

# 3rd-Party SaaS API Keys (Redacted Production Keys)
DEEPGRAM_API_KEY=dg_live_*****
GROQ_API_KEY=gsk_*****
```

19. Roadmap & Technology Milestones

Clozflow's product development is divided into seven phases, scaling the platform from its current MVP status to a fully integrated B2B sales intelligence engine.



Figure 19.1: Clozflow Multi-Phase Technology Convergence Roadmap

Development milestones focus on expanding channel integrations, improving classification models, and building enterprise-grade analytics interfaces. Retraining logs support continuous performance tuning.

20. System Telemetry: Monitoring & Logging

The Clozflow backend uses structural logging to monitor system health and AI performance. Every request is logged with its duration and scoring outputs.

We monitor three critical telemetry categories:

Telemetry Focus	Engineering Deployment Configuration
System Performance	AWS CloudWatch tracks container CPU metrics, memory utilization, and network load. Redis memory limits are monitored to manage active cache instances.
LLM API Latency & SLA Errors	Prometheus monitors LLM request durations and error rates. If 95th percentile latency exceeds 12.0 seconds, alerts trigger fallback audits.

Coaching Quality Validation	Interactions are recorded with their validation score. Responses scoring ≥ 0.85 are logged to build a dataset for subsequent model retraining, while low-scoring interactions are flagged for review.
-----------------------------	--

21. DevOps: CI/CD Delivery Pipeline

Clozflow uses automated Git-triggered pipelines to manage code deployments, run test suites, and deploy Docker containers to the staging and production environments.

The build and release cycle consists of four continuous steps:

Pipeline Step	System Actions & Validation Checks
1. Commit & Test	Developers push features to GitHub. A GitHub Actions runner triggers pytest runs, runs flake8 linter checks, and verifies model weights formats.
2. Container Build	If tests pass, the pipeline builds the FastAPI Docker container, tags the image, and uploads it to Amazon Elastic Container Registry (ECR).
3. Staging Promotion	The pipeline deploys the new image to the staging ECS task cluster, runs integration tests against a mock database, and verifies WebSocket stability.
4. Production Release	Upon authorization, the pipeline triggers a rolling update on the production ECS service task cluster. It maintains active tasks to ensure zero-downtime deployments.

22. Future Desktop & Mobile Architectures

To support sales channels outside the web browser, Clozflow's architecture is designed to accommodate future native desktop and mobile companion applications.

- Native Desktop Agent: Will leverage Electron or Rust (Tauri) to capture global audio output on Windows/macOS. This allows Clozflow to co-pilot calls on Zoom, Microsoft Teams, and native dialers by connecting directly to system sound drivers.
- Mobile Companion App: A Flutter-based mobile application is planned to sync active WhatsApp Web chats on iOS and Android. It will feature a custom keyboard extension to insert suggested responses directly into mobile messaging fields.

23. Conclusion & Technical Diligence Summary

Clozflow's decoupled hybrid architecture offers a highly scalable solution for real-time B2B sales intelligence. By combining low-latency local classifiers with vector playbooks and dynamic prompt templates, the platform delivers fast, relevant response suggestions during active negotiations.

With database schemas, WebSocket integrations, and containerized deployment structures in place, Clozflow is architected to scale from its current MVP status to a comprehensive enterprise B2B sales platform.

[END OF SPECIFICATION - DOCUMENT CLASSIFICATION: PUBLIC AUDIT EDITION]